

UART Shell Project

Nathaniel Ross

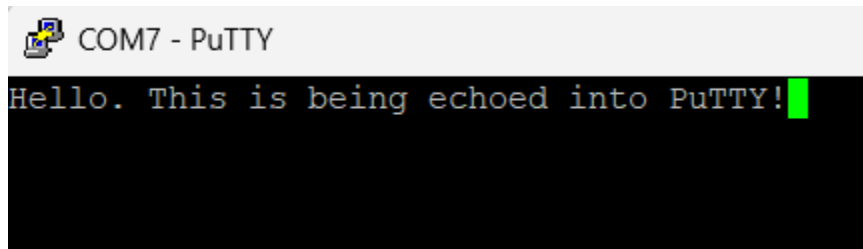
Task A: UART init + echo

Do: Write a polling loop that echoes back whatever typed in PuTTY console.

Code:

```
MX_USART2_UART_Init();
const char *message = "Hello";
uint8_t receive[1];
HAL_UART_Transmit(&huart2, (uint8_t*)message, 5, 10);
while (1)
{
    HAL_UART_Receive(&huart2, receive, 1, HAL_MAX_DELAY);
    HAL_UART_Transmit(&huart2, receive, 1, 10);
}
```

Results:



Learned:

- Current implementation uses **polling**: CPU waits for a certain amount of time and then processes an input/output. It asks “has a byte arrived?” in a loop.
- This uses a busy wait: HAL_MAX_DELAY makes the CPU wait until the next input, this wastes any additional processing capabilities for the CPU
- To solve this, we should use **interrupts**: stops whatever the CPU is doing, run a specific function, and come back. Function is called ISR (Interrupt Service Routine)

Task B: Interrupt-driven RX

Do: Use interrupts instead of polling for data transfer.

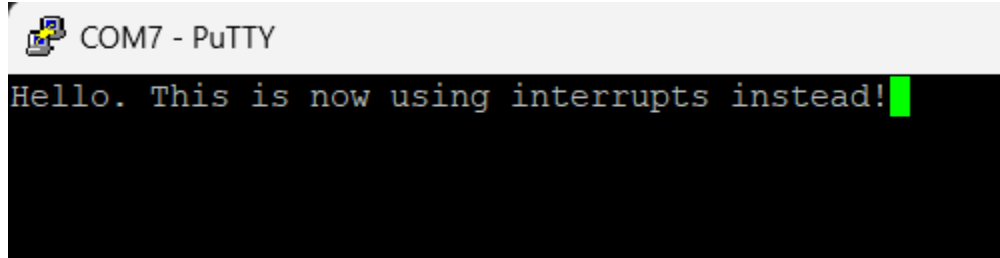
Code:

```
/* USER CODE BEGIN PV */
uint8_t receive[1];

/* USER CODE BEGIN 2 */
const char *message = "Hello";
HAL_UART_Transmit(&huart2, (uint8_t*)message, 5, 10);
HAL_UART_Receive_IT(&huart2, receive, 1);

/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART2)
    {
        HAL_UART_Transmit_IT(huart, receive, 1);
        HAL_UART_Receive_IT(huart, receive, 1);
    }
}
```

Results:



Learned:

- Used interrupts to transmit messages. Now, the while(1) loop in main is empty, meaning the CPU has processing power to do other tasks
- Difference between Receive and Receive_IT: The first uses blocking (polling) mode and second uses non-blocking (interrupts) mode
- Receive_IT arms one byte for an interrupt, RxCpltCallback is a callback that uses the byte and echoes it, then re-arms the byte for another interrupt with Receive_IT
- In callback function, UART instance (USART2) is guarded from any other UART interrupt

Task C: Ring Buffer

Do: Create a ring buffer to store all bytes arriving from interrupt.

Code:

```
/* USER CODE BEGIN PTD */
#define BUFFER_SIZE 64

/* USER CODE BEGIN PV */
uint8_t receive[1];
uint8_t buffer[BUFFER_SIZE];
volatile uint8_t head;
volatile uint8_t tail;

/* Private function prototypes
-----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
void ring_write(uint8_t byte);
uint8_t ring_read(void);
uint8_t ring_empty(void);

/* USER CODE BEGIN 2 */
head = 0;
tail = 0;
HAL_UART_Receive_IT(&huart2, receive, 1);

/* USER CODE BEGIN WHILE */
while (1)
{
    if(!ring_empty())
    {
        uint8_t byte = ring_read();
        HAL_UART_Transmit_IT(&huart2, &byte, 1);
    }
}

/* USER CODE BEGIN 4 */
void ring_write(uint8_t byte)
{
    // checks if buffer is not full
    if (head != (tail - 1 + BUFFER_SIZE) % BUFFER_SIZE)
    {
        // writes to head of ring buffer
        buffer[head] = byte;
        head = (head + 1) % BUFFER_SIZE; // wrap-around
    }
}
```

```

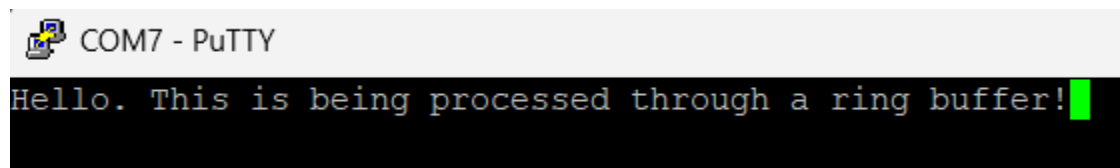
uint8_t ring_read(void)
{
    if (head == tail) // checks if empty buffer
    {
        return 0;
    }
    // reads current byte (tail) and advances
    else
    {
        uint8_t byte = buffer[tail];
        tail = (tail + 1) % BUFFER_SIZE; // wrap-around
        return byte;
    }
}

uint8_t ring_empty(void)
{
    // returns true if ring is empty
    if (head == tail)
        return 1;
    else
        return 0;
}

// callback function
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // checks for USART 2
    if(huart->Instance == USART2)
    {
        ring_write(receive[0]);
        HAL_UART_Receive_IT(huart, receive, 1);
    }
}

```

Results:



Learned:

- Ring buffer used to manage fixed size, continuous stream of data. ISR routine doesn't block. This could be typing in the command prompt or a stream of LED commands
- Main loop uses buffer at its own pace, receiving and processing aren't mixed
- `buffer` sacrifices a slot to determine if it is full (so head doesn't wrap into tail)
- Full and empty ring conditions are handled, wrap-around feature handled for O(1) ops
- `volatile` keyword used to declare head and tail variables because they are used for interrupts, so they cannot be cached / optimized by the compiler

Task D: Line parser

Do: Create a line parser that accumulates bytes until a complete command arrives.

Code:

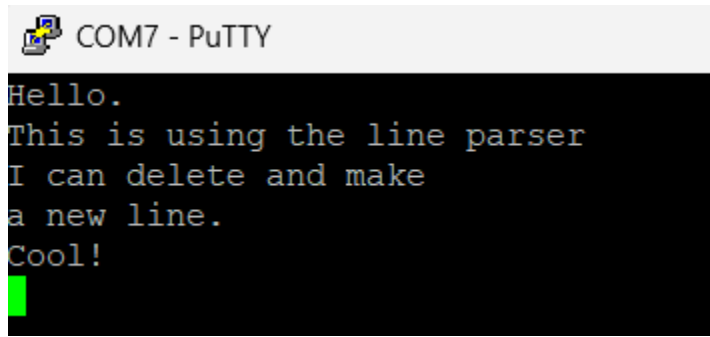
```
#define CMD_SIZE 32

/* USER CODE BEGIN 1 */
uint8_t cmd_buff[CMD_SIZE];
uint8_t cmd_index = 0;

while (1)
{
    if(!ring_empty())
    {
        uint8_t byte = ring_read();

        // New line
        if(byte == '\r' || byte == '\n')
        {
            cmd_buff[cmd_index] = '\0';
            uint8_t newline[] = "\r\n";
            HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);
            cmd_index = 0;
            memset(cmd_buff, 0, CMD_SIZE);
        }
        // Backspace
        else if(byte == '\b' && cmd_index != 0)
        {
            cmd_index--;
            cmd_buff[cmd_index] = 0;
            uint8_t bs_seq[] = "\b \b";
            HAL_UART_Transmit(&huart2, bs_seq, 3, HAL_MAX_DELAY);
        }
        // Normal character, buffer isn't full
        else if(cmd_index < CMD_SIZE - 1)
        {
            cmd_buff[cmd_index] = byte;
            cmd_index++;
            HAL_UART_Transmit(&huart2, &byte, 1, HAL_MAX_DELAY);
        }
    }
}
```

Results:



```
COM7 - PuTTY
Hello.
This is using the line parser
I can delete and make
a new line.
Cool!
█
```

Learned:

- Line accumulation into buffer
- Can detect inputs, will be used in the next step to trigger commands
- Buffer overflow protection
- Clean buffer reset after command is sent
- This changed TX (transmission) to blocking (polling). This is fine because transmission only happens in response to user input (typing character and then echo). We would care about interrupt-driven TX if streaming large amounts of data continuously or had timing-critical work to do for transmission. For typing, transmitting one byte takes 86us at 115200 baud, so timing is a nonfactor.
- RX (receiving) is still interrupt driven

Task E: Command dispatcher

Do: Create a command dispatcher that, after newline case, compares the input to commands

Code:

```
/* USER CODE BEGIN 4 */
void process_command(uint8_t* cmd)
{
    uint8_t newline[] = "\r\n";
    HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);

    if(strcmp((char*)cmd, "led on") == 0)
    {
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
        led_state = 1;
        uint8_t msg[] = "LED ON\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
    else if(strcmp((char*)cmd, "led off") == 0)
    {
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
        led_state = 0;
        uint8_t msg[] = "LED OFF\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
    else if(strcmp((char*)cmd, "status") == 0)
    {
        if(led_state)
        {
            uint8_t msg[] = "LED is ON\r\n";
            HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1,
HAL_MAX_DELAY);
        }
        else
        {
            uint8_t msg[] = "LED is OFF\r\n";
            HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1,
HAL_MAX_DELAY);
        }
    }
    else
    {
        uint8_t msg[] = "unknown cmd\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
}
```

Results:

```
status
LED is ON
led off
LED OFF
status
LED is OFF
█
```

Learned:

- How to create a strcmp-based state machine that executes real hardware actions such as turning on or off an LED
- One bug: If backspace is used, the command will not work. This can be fixed by sanitizing the input of any backspaces
- This can have much more possibilities, such as:
 - “help” command to list available commands
 - “led toggle” to toggle the led from its current state
 - “uptime” to see how long the LED has been on with HAL_GetTick()
 - Be able to control output pins on the STM32 to turn on external devices connected via breadboard. This could include changing the brightness of an LED with PWM, turning on a fan, typing and displaying to a LCD

What was learned overall:

- Difference between interrupt and polling
- Interrupt-driven UART RX (bytes arrive via ISR, don't block CPU)
- Ring buffer (decouples ISR from main loop, handles data burst safely)
- Line parser (accumulate characters, detect command completion)
- Command dispatcher (compare command via state machine to manipulate hardware)